

Three-Tier Application Deployment with AWS, Terraform, and Jenkins

Background

Your organization has developed a three-tier web application consisting of:

1. Frontend: A web-based user interface built with React.js
2. Backend: A REST API service built with Node.js
3. Database: MongoDB for data persistence

The development team has completed the application code, but it requires proper infrastructure setup and deployment configuration. The application code is stored in a Git repository, and it requires environment-specific configurations:

- Backend needs a .env file with MONGO_URI and PORT variables
- Frontend needs the backend IP configured in /src/url.js

MyTask

As a DevOps engineer, you are tasked with implementing a complete infrastructure and deployment pipeline using:

- Infrastructure as Code: AWS resources using Terraform (including instance configuration)
- CI/CD Pipeline: Automated deployment using Jenkins

You must create the necessary scripts and configurations to provision, configure, and deploy the three-tier application in AWS while ensuring proper communication between all components.

1. Install Terraform and awscli (on macbook)

brew tap hashicorp/tap

brew install hashicorp/tap/terraform

brew install awscli

```
abhutoria@abhutor-ltmbqz4 terraform %  
abhutoria@abhutor-ltmbqz4 terraform % mkdir three-tier-app-deployment  
abhutoria@abhutor-ltmbqz4 terraform % cd three-tier-app-deployment  
abhutoria@abhutor-ltmbqz4 three-tier-app-deployment % awscli --version  
zsh: command not found: awscli  
abhutoria@abhutor-ltmbqz4 three-tier-app-deployment %  
abhutoria@abhutor-ltmbqz4 three-tier-app-deployment % brew install awscli  
==> Auto-updating Homebrew...  
Adjust how often this is run with HOMEBREW_AUTO_UPDATE_SECS or disable with  
HOMEBREW_NO_AUTO_UPDATE. Hide these hints with HOMEBREW_NO_ENV_HINTS (see `man brew`).  
==> Auto-updated Homebrew!  
Updated 2 taps (homebrew/core and homebrew/cask).  
==> New Formulae  
algolia                cloudfoundry-cli      globstar               immich-go             targetiffertools      nom  
async_simple           dvr-scan              gotz                   kafkactl              lazyjournal           org-formation  
bpmnlint               fortitude             hisstory              kapp                  mox                    ov  
cf-terraforming        gersemi               i686-elf-grub          kirimase               netfetch              pivy  
==> New Casks  
candy-crisis           font-sf-mono-nerd-font-ligaturized  macskk  
consul                 font-winky-sans        nvidia-nsight-compute  
focu                   fuse-t                  opera-air  
  
You have 5 outdated formulae installed.  
  
==> Downloading https://ghcr.io/v2/homebrew/core/awscli/manifests/2.24.20  
#####  
==> Fetching dependencies for awscli: pycparser, cffi, ca-certificates, cryptography, sqlite and python@3.12  
==> Downloading https://ghcr.io/v2/homebrew/core/pycparser/manifests/2.22.1  
#####  
==> Fetching pycparser  
==> Downloading https://ghcr.io/v2/homebrew/core/pycparser/blobs/sha256:96eddd22a812be4f919562d6525ab8fa19b03b706  
#####  
==> Downloading https://ghcr.io/v2/homebrew/core/cffi/manifests/1.17.1_1  
#####  
==> Fetching cffi  
==> Downloading https://ghcr.io/v2/homebrew/core/cffi/blobs/sha256:908333c6c31b4da4876ccb707cdf2b1ec52aee3d83fc3c  
#####  
==> Downloading https://ghcr.io/v2/homebrew/core/ca-certificates/manifests/2025-02-25  
#####  
==> Fetching ca-certificates  
==> Downloading https://ghcr.io/v2/homebrew/core/ca-certificates/blobs/sha256:8f46fb05f753f80f2bc398c0a24b10d680b  
#####  
==> Downloading https://ghcr.io/v2/homebrew/core/cryptography/manifests/44.0.2  
#####  
==> Fetching cryptography  
==> Downloading https://ghcr.io/v2/homebrew/core/cryptography/blobs/sha256:3c0211c9200c69fd4502e667237724d03d527c  
#####  
==> Downloading https://ghcr.io/v2/homebrew/core/sqlite/manifests/3.49.1  
#####  
==> Fetching sqlite
```

```

abhutoria@abhutor-ltmbqz4 three-tier-app-deployment % terraform --version
Terraform v1.10.5
on darwin_arm64

Your version of Terraform is out of date! The latest version
is 1.11.1. You can update by downloading from https://www.terraform.io/downloads.html
abhutoria@abhutor-ltmbqz4 three-tier-app-deployment % brew install awscli
Warning: awscli 2.24.20 is already installed and up-to-date.
To reinstall 2.24.20, run:
  brew reinstall awscli

```

> Run command “**aws configure**” to store access keyID under *~/aws/credentials*.

Access keys (1)

Create access key

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

AKIA6GBMCU7ZLAVBF3XM Description - Last used 26 days ago Last used region us-east-1	Status Active Created 60 days ago Last used service sts	Actions
--	---	--------------------

> Create respective terraform and script files to run.

2. Deploy the infrastructure with Terraform

a. Initialize Terraform:

```

abhutoria@abhutor-ltmbqz4 % terraform %
abhutoria@abhutor-ltmbqz4 % terraform % ls
compute.tf      main.tf      outputs.tf      scripts      security.tf      terraform.tfvars      variables.tf
abhutoria@abhutor-ltmbqz4 % terraform % pwd
/Users/abhutoria/heroVired/terraform/three-tier-app-deployment/terraform
abhutoria@abhutor-ltmbqz4 % terraform %
abhutoria@abhutor-ltmbqz4 % terraform %
abhutoria@abhutor-ltmbqz4 % terraform %
abhutoria@abhutor-ltmbqz4 % terraform % terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching ">= 5.0"...
- Installing hashicorp/aws v5.90.0...
- Installed hashicorp/aws v5.90.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
abhutoria@abhutor-ltmbqz4 % terraform %

```

b. Validate the Terraform configuration:

```
abhutoria@abhutor-ltmbqz4 terraform % terraform validate
Success! The configuration is valid.

abhutoria@abhutor-ltmbqz4 terraform %
```

c. Generate and review the execution plan:

```
abhutoria@abhutor-ltmbqz4 terraform % terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.backend will be created
+ resource "aws_instance" "backend" {
  + ami              = "ami-0e1bed4f06a3b463d"
  + arn              = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone = (known after apply)
  + cpu_core_count   = (known after apply)
  + cpu_threads_per_core = (known after apply)
  + disable_api_stop  = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized     = (known after apply)
  + enable_primary_ipv6 = (known after apply)
  + get_password_data = false
  + host_id           = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile = (known after apply)
  + id                = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle = (known after apply)
  + instance_state     = (known after apply)
  + instance_type      = "t2.small"
  + ipv6_address_count = (known after apply)
  + ipv6_addresses     = (known after apply)
  + key_name           = "EC2-AMI-Aks-HV"
  + monitoring         = (known after apply)
  + outpost_arn        = (known after apply)
  + password_data      = (known after apply)
  + placement_group    = (known after apply)
  + placement_partition_number = (known after apply)
  + primary_network_interface_id = (known after apply)
  + private_dns        = (known after apply)
  + private_ip         = (known after apply)
  + public_dns         = (known after apply)
  + public_ip          = (known after apply)
  + secondary_private_ips = (known after apply)
  + security_groups    = (known after apply)
```

d. Apply the Terraform configuration:


```

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_security_group.backend_sg: Creating...
aws_security_group.frontend_sg: Creating...
aws_security_group.frontend_sg: Creation complete after 5s [id=sg-08cd3a30494c775a3]
aws_security_group.backend_sg: Creation complete after 5s [id=sg-0240c177c8546cdcc]
aws_security_group.mongodb_sg: Creating...
aws_security_group.mongodb_sg: Creation complete after 5s [id=sg-0aff3ff8040d1df6d]
aws_instance.mongodb: Creating...
aws_instance.mongodb: Still creating... [10s elapsed]
aws_instance.mongodb: Creation complete after 16s [id=i-091506ed058d4226a]
aws_instance.backend: Creating...
aws_instance.backend: Still creating... [10s elapsed]
aws_instance.backend: Creation complete after 15s [id=i-070bc77d7f38ff2cd]
aws_instance.frontend: Creating...
aws_instance.frontend: Still creating... [10s elapsed]
aws_instance.frontend: Creation complete after 16s [id=i-0c81fceb796f90741]

Apply complete! Resources: 6 added, 0 changed, 0 destroyed.

Outputs:

application_url = "http://35.168.16.78"
backend_private_ip = "172.31.46.39"
backend_public_ip = "35.174.11.82"
frontend_public_ip = "35.168.16.78"
mongodb_private_ip = "172.31.32.116"
abhutoria@abhutor-ltmbqz4 terraform %

```

e. Verified URL to run, <http://35.168.16.78>



3. Verify the deployment

a. Check the MongoDB instance:

```
abhutoria@abhutor-1tmbqz4 heroVired %
abhutoria@abhutor-1tmbqz4 heroVired % ssh -i "EC2-AMI-Aks-HV.pem" ubuntu@ec2-34-229-118-92.compute-1.amazonaws.com
The authenticity of host 'ec2-34-229-118-92.compute-1.amazonaws.com (100.64.1.137)' can't be established.
ED25519 key fingerprint is SHA256:A8txyvBZZS5t6+7wWIgp9QfWhhX0nqQ1DyAWaqkHKfo.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-34-229-118-92.compute-1.amazonaws.com' (ED25519) to the list of known hosts.
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1021-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Mar  8 10:48:47 UTC 2025

System load:  0.08          Processes:      102
Usage of /:   25.7% of 7.57GB Users logged in:  0
Memory usage: 13%          IPv4 address for eth0: 172.31.32.116
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

9 updates can be applied immediately.
9 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-32-116:~$
```

```
root@ip-172-31-32-116:/etc/apt/sources.list.d# ls
mongodb-org-6.0.list
root@ip-172-31-32-116:/etc/apt/sources.list.d# sudo systemctl start mongod
root@ip-172-31-32-116:/etc/apt/sources.list.d#
root@ip-172-31-32-116:/etc/apt/sources.list.d# sudo systemctl enable mongod
Created symlink /etc/systemd/system/multi-user.target.wants/mongod.service → /lib/systemd/system/mongod.service.
root@ip-172-31-32-116:/etc/apt/sources.list.d#
root@ip-172-31-32-116:/etc/apt/sources.list.d# sudo systemctl status mongod
● mongod.service - MongoDB Database Server
   Loaded: loaded (/lib/systemd/system/mongod.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2025-03-08 11:00:59 UTC; 11s ago
     Docs: https://docs.mongodb.org/manual
    Main PID: 12101 (mongod)
      Memory: 69.6M
         CPU: 769ms
    CGroup: /system.slice/mongod.service
            └─12101 /usr/bin/mongod --config /etc/mongod.conf

Mar 08 11:00:59 ip-172-31-32-116 systemd[1]: Started MongoDB Database Server.
Mar 08 11:00:59 ip-172-31-32-116 mongod[12101]: {"t":{"$date":"2025-03-08T11:00:59.850Z"},"s":"I",  "c":"CONTROL",  "id":7484600, "ctx":"","msg":"Environment variable MONGODB_CONFIG_OVERRID
root@ip-172-31-32-116:/etc/apt/sources.list.d#
```

The problem faced by me during MongoDB is, it was not installed properly.
Followed below steps:

Import the MongoDB public key

```
wget -qO - https://www.mongodb.org/static/pgp/server-6.0.asc | sudo apt-key add -
```

```
# Add the MongoDB repository (for Ubuntu 22.04)
```

```
echo "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu  
jammy/mongodb-org/6.0 multiverse" | sudo tee  
/etc/apt/sources.list.d/mongodb-org-6.0.list
```

```
# Update package lists
```

```
sudo apt update
```

```
#Install MongoDB
```

```
sudo apt install -y mongodb-org
```

```
#Start/Enable and verify status:
```

```
sudo systemctl start mongod
```

```
sudo systemctl enable mongod
```

```
sudo systemctl status mongod
```

```
root@ip-172-31-32-116:/etc/apt/sources.list.d#  
[root@ip-172-31-32-116:/etc/apt/sources.list.d# mongod --version  
db version v6.0.20  
Build Info: {  
  "version": "6.0.20",  
  "gitVersion": "ef4efd469fb0c5626c629c4f0f21398c9b5dd08f",  
  "opensslVersion": "OpenSSL 3.0.2 15 Mar 2022",  
  "modules": [],  
  "allocator": "tcmalloc",  
  "environment": {  
    "distmod": "ubuntu2204",  
    "distarch": "x86_64",  
    "target_arch": "x86_64"  
  }  
}  
root@ip-172-31-32-116:/etc/apt/sources.list.d#
```

b. Check the Backend instance

```
root@ip-172-31-46-39:/opt/travel-memory/backend#
root@ip-172-31-46-39:/opt/travel-memory/backend# pm2 status
```

id	name	namespace	version	mode	pid	uptime	⤴	status	cpu	mem	user	watching
0	backend	default	1.0.0	fork	10416	27s	0	online	0%	63.4mb	root	disabled

```
root@ip-172-31-46-39:/opt/travel-memory/backend#
```

c. Check the Frontend instance

Travel Memory

Add Experience

Trip Name

Add your Trip Name

Trip Date

dd/mm/yyyy

dd/mm/yyyy

Name of Hotels

Add your Hotel Name

Trip Type

Select One

Total Cost

0

Places Visited

Delhi, Paris, London, etc.

Featured Trip?

☐ True

☒ False

Image Link

http://xyz.com/image.png

Short Description

Write Short Description

Experience

Write Complete Details about your experience in the trip.

CI/CD Pipeline using Jenkins

Install Jenkins/Java

Attached Jenkins-script.sh to run
And added Jenkinsfile for pipeline.

Getting Started

Create First Admin User

Username

Password

Confirm password

Full name

E-mail address

Jenkins 2.492.1

[Skip and continue as admin](#)[Save and Continue](#)

Getting Started

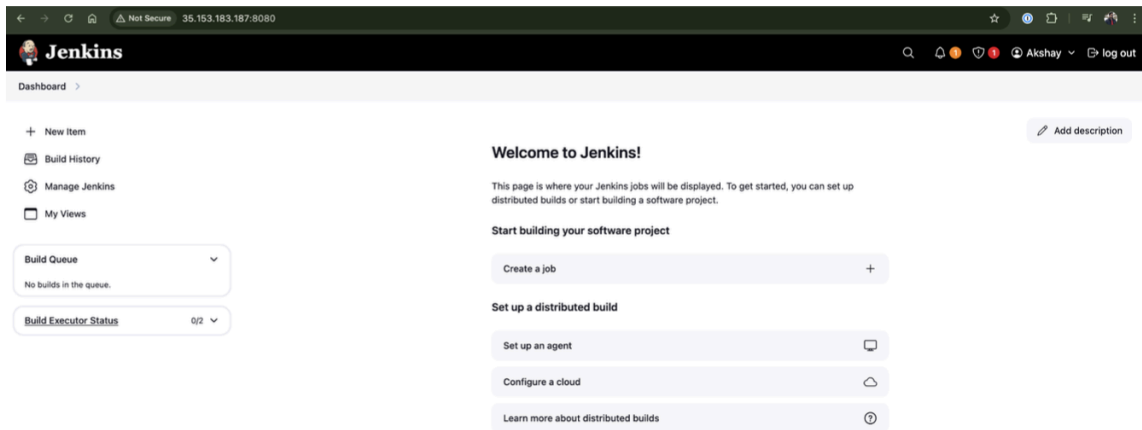
Instance Configuration

Jenkins URL:

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.

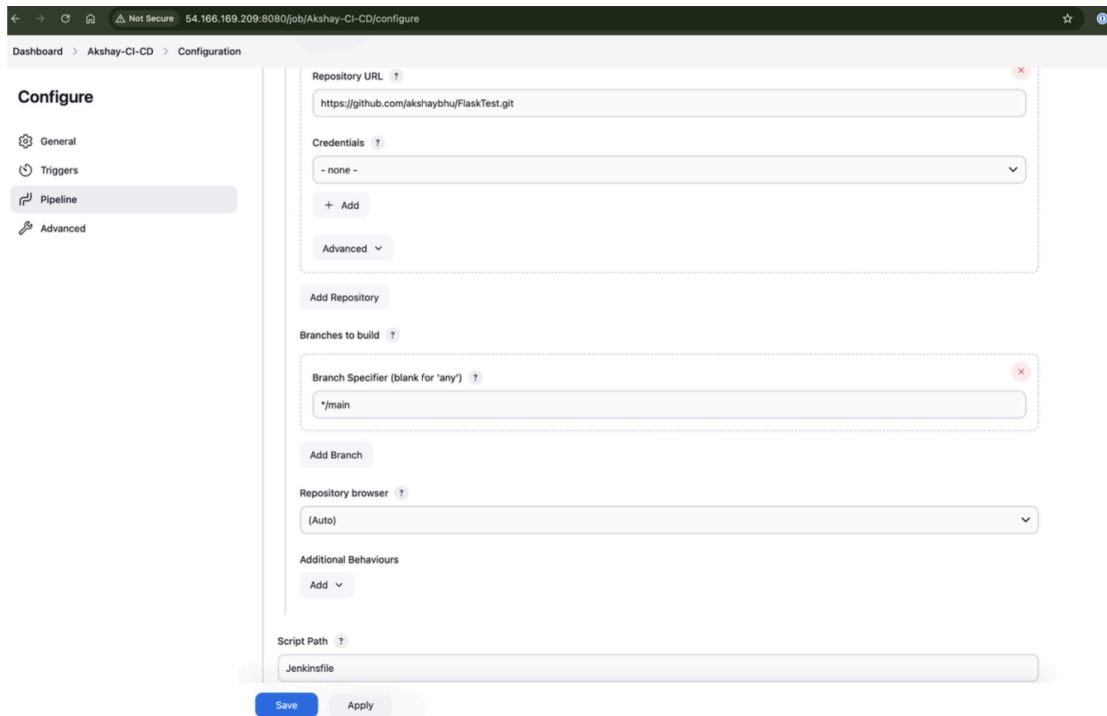
The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

Installation finished



> Moved default port of 5000 to 5010 due to mac use of this port.

In the Jenkins web UI, create a new item (a "Pipeline" project is recommended).
Configure the pipeline. You can use a Pipeline script (Jenkinsfile) for this.



To destroy using terraform use “**terraform destroy**” command

```
abhutoria@abhutor-1tmbqz4 terraform % terraform destroy
aws_security_group.backend_sg: Refreshing state... [id=sg-0240c177c8546cdcc]
aws_security_group.frontend_sg: Refreshing state... [id=sg-08cd3a30494c775a3]
aws_security_group.mongodb_sg: Refreshing state... [id=sg-0aff3ff8040d1df6d]
aws_instance.mongodb: Refreshing state... [id=i-091506ed058d4226a]
aws_instance.backend: Refreshing state... [id=i-070bc77d7f38ff2cd]
aws_instance.frontend: Refreshing state... [id=i-0c81fceb796f90741]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

# aws_instance.backend will be destroyed
- resource "aws_instance" "backend" {
  - ami                  = "ami-0e1bed4f06a3b463d" -> null
  - arn                  = "arn:aws:ec2:us-east-1:975050024946:instance/i-070bc77d7f38ff2cd" -> null
  - associate_public_ip_address = true -> null
  - availability_zone      = "us-east-1a" -> null
  - cpu_core_count         = 1 -> null
  - cpu_threads_per_core   = 1 -> null
  - disable_api_stop       = false -> null
  - disable_api_termination = false -> null
  - ebs_optimized          = false -> null
  - get_password_data      = false -> null
  - hibernation            = false -> null
  - id                    = "i-070bc77d7f38ff2cd" -> null
  - instance_initiated_shutdown_behavior = "stop" -> null
  - instance_state         = "running" -> null
  - instance_type          = "t2.small" -> null
  - ipv6_address_count      = 0 -> null
  - ipv6_addresses         = [] -> null
  - key_name               = "EC2-AMI-Aks-HV" -> null
  - monitoring              = false -> null
  - placement_partition_number = 0 -> null
  - primary_network_interface_id = "eni-0b467067e6237f7ab" -> null
  - private_dns             = "ip-172-31-46-39.ec2.internal" -> null
  - private_ip             = "172.31.46.39" -> null
  - public_dns             = "ec2-35-174-11-82.compute-1.amazonaws.com" -> null
  - public_ip              = "35.174.11.82" -> null
  - secondary_private_ips   = [] -> null
  - security_groups        = [
    - "backend-sg",
  ] -> null
  - source_dest_check       = true -> null
```

Plan: 0 to add, 0 to change, 0 to destroy.

Changes to Outputs:

- application_url = "http://35.168.16.78" -> null
- backend_private_ip = "172.31.46.39" -> null
- backend_public_ip = "35.174.11.82" -> null
- frontend_public_ip = "35.168.16.78" -> null
- mongodb_private_ip = "172.31.32.116" -> null

Do you really want to destroy all resources?

Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

```
aws_instance.frontend: Destroying... [id=i-0c81fceb796f90741]
aws_instance.frontend: Still destroying... [id=i-0c81fceb796f90741, 10s elapsed]
aws_instance.frontend: Still destroying... [id=i-0c81fceb796f90741, 20s elapsed]
aws_instance.frontend: Destruction complete after 21s
aws_security_group.frontend_sg: Destroying... [id=sg-08cd3a30494c775a3]
aws_instance.backend: Destroying... [id=i-070bc77d7f38ff2cd]
aws_security_group.frontend_sg: Still destroying... [id=sg-08cd3a30494c775a3, 10s elapsed]
aws_instance.backend: Still destroying... [id=i-070bc77d7f38ff2cd, 10s elapsed]
aws_instance.backend: Still destroying... [id=i-070bc77d7f38ff2cd, 20s elapsed]
aws_security_group.frontend_sg: Still destroying... [id=sg-08cd3a30494c775a3, 20s elapsed]
aws_instance.backend: Destruction complete after 22s
aws_instance.mongodb: Destroying... [id=i-091506ed058d4226a]
aws_security_group.frontend_sg: Still destroying... [id=sg-08cd3a30494c775a3, 30s elapsed]
aws_instance.mongodb: Still destroying... [id=i-091506ed058d4226a, 10s elapsed]
aws_security_group.frontend_sg: Still destroying... [id=sg-08cd3a30494c775a3, 40s elapsed]
aws_instance.mongodb: Still destroying... [id=i-091506ed058d4226a, 20s elapsed]
aws_security_group.frontend_sg: Still destroying... [id=sg-08cd3a30494c775a3, 50s elapsed]
aws_instance.mongodb: Still destroying... [id=i-091506ed058d4226a, 30s elapsed]
aws_instance.mongodb: Destruction complete after 34s
aws_security_group.mongodb_sg: Destroying... [id=sg-0aff3ff8040d1df6d]
aws_security_group.frontend_sg: Still destroying... [id=sg-08cd3a30494c775a3, 1m0s elapsed]
aws_security_group.mongodb_sg: Still destroying... [id=sg-0aff3ff8040d1df6d, 10s elapsed]
aws_security_group.frontend_sg: Still destroying... [id=sg-08cd3a30494c775a3, 1m10s elapsed]
aws_security_group.mongodb_sg: Still destroying... [id=sg-0aff3ff8040d1df6d, 20s elapsed]
aws_security_group.frontend_sg: Still destroying... [id=sg-08cd3a30494c775a3, 1m20s elapsed]
aws_security_group.mongodb_sg: Still destroying... [id=sg-0aff3ff8040d1df6d, 30s elapsed]
aws_security_group.frontend_sg: Still destroying... [id=sg-08cd3a30494c775a3, 1m30s elapsed]
aws_security_group.mongodb_sg: Still destroying... [id=sg-0aff3ff8040d1df6d, 40s elapsed]
aws_security_group.frontend_sg: Still destroying... [id=sg-08cd3a30494c775a3, 1m40s elapsed]
aws_security_group.mongodb_sg: Still destroying... [id=sg-0aff3ff8040d1df6d, 50s elapsed]
aws_security_group.frontend_sg: Still destroying... [id=sg-08cd3a30494c775a3, 1m50s elapsed]
aws_security_group.mongodb_sg: Still destroying... [id=sg-0aff3ff8040d1df6d, 1m0s elapsed]
```